CS4032 – Web Programming (Spring 2026)

BS(CS) 6E

Assignment# 4

LATE SUBMISSION & PEER PLAGIARISM WILL RESULT IN 0

Submission Instructions:

1. Submit code zip files and the word file containing all source code.
2. Each question must be in a separate folder with its own package.json.
3. Do NOT include the **node_modules** folder in your zip submission and must upload **GitHub Repo**.
4. Test every API endpoint using Postman or Thunder Client and include screenshots of each request and response in a PDF report.
5. Late submissions are not allowed.

Evaluation Criteria:

- Proper file naming, folder structure (routes, controllers, models), and well-organized JavaScript code.
- Correct implementation of all required endpoints and strict adherence to the given specifications.
- Working API endpoints —testing in Postman/Thunder Client.

==========

Instructions

- Use Node.js with the Express framework for all questions.
- Use MongoDB via Mongoose for database operations.
- Apply proper HTTP status codes (200, 201, 400, 404, 500, etc.) and return responses in JSON format.
- Implement input validation and proper error handling in every endpoint.
- Code must be clean, readable, and modular (separate routes, controllers, and models).

Objective

This assignment evaluates your ability to:

- Design RESTful API endpoints following industry conventions.
- Build and connect to a MongoDB database using Mongoose schemas.
- Download MongoDB Compass to see Outputs.
- Implement complete CRUD operations with validation and error handling.
- Establish relationships between MongoDB collections using ObjectId references and population.

==========

SCENARIO: "University & Blogging Platform Backend"

You are required to build two REST APIs using Node.js, Express, and MongoDB. The first manages student records for a university, and the second powers a blogging platform where users post articles and others comment on them.

TASK 1: Student Management System API (50 Marks)

Build a RESTful API to manage student records for a university. The API must allow creating, reading, updating, and deleting student data stored in a MongoDB database. Implement all CRUD operations along with search, filtering, and pagination.

Schema Requirements

Create a Student schema with the following fields:

- rollNumber (String, required, unique)
- name (String, required)
- email (String, required, unique, valid email format)
- department (String, required, e.g., 'Computer Science', 'Electrical')
- cgpa (Number, between 0.0 and 4.0)
- enrollmentYear (Number, required)
- isActive (Boolean, default: true)

Required API Endpoints

- **`POST`** `/api/students` — Add a new student to the database. Validate all required fields before saving.
- **`GET`** `/api/students` — Retrieve all students. Support optional query parameters for filtering by department and pagination (page, limit).
- **`GET`** `/api/students/:id` — Retrieve a single student by their MongoDB _id. Return 404 if not found.
- **`GET`** `/api/students/search?name=...` — Search students by name (case-insensitive, partial match using regex).
- **`PUT`** `/api/students/:id` — Update all fields of an existing student record.
- **`PATCH`** `/api/students/:id` — Update one or more fields of a student record (partial update).
- **`DELETE`** `/api/students/:id` — Permanently delete a student from the database.
- **`PATCH`** `/api/students/:id/deactivate` — Soft delete by setting isActive to false instead of removing the record.

Sample Request Body (POST)

```
{
  "rollNumber": "21-CS-105",
  "name": "Ali Hassan",
  "email": "ali.hassan@university.edu",
  "department": "Computer Science",
  "cgpa": 3.78,
  "enrollmentYear": 2021
}
```

Marks Distribution

- Project setup, schema design, and database connection — 10 marks
- Implementation of all CRUD endpoints — 15 marks
- Search, filtering, and pagination — 10 marks
- Validation, error handling, and proper status codes — 15 marks

TASK 2: Blog Application API with Relationships (50 Marks)

Develop a RESTful API for a simple blogging platform where users can write posts and add comments to those posts. This task tests your understanding of relationships between MongoDB collections using references (ObjectId) and the populate method.

Schema Requirements

User Schema: username (String, required, unique), email (String, required, unique), password (String, required, min length 6), createdAt (Date, default: now).

Post Schema: title (String, required), content (String, required), author (ObjectId, ref: 'User', required), tags (Array of Strings), createdAt (Date, default: now).

Comment Schema: text (String, required), post (ObjectId, ref: 'Post', required), user (ObjectId, ref: 'User', required), createdAt (Date, default: now).

Required API Endpoints

User Endpoints

- `POST /api/users/register` — Register a new user. Hash the password before storing using bcrypt.
- `GET /api/users` — Retrieve all users (exclude the password field from the response).
- `GET /api/users/:id` — Get a single user by ID along with all the posts they have written (use populate).

Post Endpoints

- `POST /api/posts` — Create a new post. The author field must reference an existing user.
- `GET /api/posts` — Retrieve all posts with author details populated (username and email only).
- `GET /api/posts/:id` — Retrieve a single post with author details and all its comments populated.
- `GET /api/posts/tag/:tag` — Retrieve all posts that contain the given tag.
- `PUT /api/posts/:id` — Update a post's title, content, or tags.
- `DELETE /api/posts/:id` — Delete a post and also remove all comments associated with it.

Comment Endpoints

- `POST /api/posts/:postId/comments` — Add a comment to a specific post. Validate that the post and user both exist.
- `GET /api/posts/:postId/comments` — Get all comments for a specific post (populate the user field).
- `DELETE /api/comments/:id` — Delete a specific comment by its ID.

Sample Request Body (Create Post)

```
{
  "title": "Getting Started with REST APIs",
  "content": "REST stands for Representational State Transfer...",
  "author": "65f1a2b8c9d4e5f6a7b8c9d0",
  "tags": ["nodejs", "express", "mongodb"]
}
```

Marks Distribution

- Schema design with proper references between collections — 10 marks
- User registration with password hashing and user endpoints — 10 marks
- Post CRUD operations with population — 10 marks
- Comment endpoints and cascade deletion of comments — 10 marks
- Error handling, validation, and code quality — 10 marks

Submission Guidelines

- Submit a single zipped folder named: YourName_RollNumber_BackendAssignment.zip
- Include both projects (Q1 and Q2) in separate sub-folders, each with its own package.json.
- Do NOT include the **node_modules** folder in the submission.
- Provide a PDF report with Postman screenshots of every endpoint request and response.
- Include GitHub Repo and the README.md.
- Late submissions are not allowed.

Helping Material:

https://www.geeksforgeeks.org/node-js/how-to-build-a-restful-api-using-node-express-and-mongodb/

https://www.youtube.com/watch?v=9OfL9H6AmhQtps://youtu.be/fYTTUBa-lPc?si=Y_JpPYwcJzEztwEx